

Islamic University of Technology

EEE 4416 Mini-Project

Name: Shaheen Rahman Dibya

ID: 230021315

Section: C

Department: EEE

Project Name: Golf Streak Game

Date of Submission: 16/9/2026

1. Introduction

For this project, an interactive GUI application was chosen to build the game called **Golf Streak** rather than taking the standard data analysis route. It is a 2D mini-golf game built entirely in MATLAB using standard figure and control functions, which gave me direct control over every callback, redraw, and animation frame. Instead of a simple menu with a few static buttons, the game runs a smooth real-time loop complete with continuous physics, two distinct ways to take a shot, and a 10-level campaign that generates fresh, randomized layouts every time we play.

2. Objective

The main goal of this project is to create a responsive, event-driven game in MATLAB while handling all graphics and timing manually. Specifically, the project aims to implement realistic 2D physics to handle ball friction, surface interactions, and wall reflections. It also focuses on procedurally generating fair and playable levels from scratch, giving players the freedom to shoot using either precise dashboard sliders or free-form mouse dragging, and offering visual feedback through a live dashboard and particle effects.

3. Key Features

The feature set below is what distinguishes Golf Streak from a minimal, few-widget GUI:

Feature	Description
10-level campaign	Each level is built procedurally on the fly, so the layout changes every time you play. As we advance through the game, the challenge gradually ramps up and the hole gets smaller, par limits adjust, and more obstacles are tossed into our path.
Dynamic boundary system	1–3 of the 4 outer edges are randomly chosen as solid "bounce walls" each level and the rest are open hazards.
Realistic ball physics	Velocity-based motion with surface-dependent friction and energy-losing wall reflections.
Dual control scheme	Aim/power sliders or mouse drag; both feed the same shot pipeline.
Predictive trajectory preview	A dotted raycast line previews the shot path, including one wall bounce, before every stroke.

Feature	Description
Fairness-checked hazards	Sand and water are placed randomly but never on the start point or blocking the hole.
Checkpoint recovery	The last valid position is remembered, so an out-of-bounds shot or water hazard doesn't restart the level.
Club selection	Driver, Putter, and Sand Wedge each apply a different power multiplier and bunker friction.
Particle effects	Sand kicks, wall-bounce sparks, hazard splashes, and a hole-in-one celebration, all through a lightweight particle system.
Spare-shot economy	One budget of 10 spare shots carries across all 10 levels, turning the campaign into a survival-style challenge.

3. System Architecture

The game operates within a single figure window split into two main areas. The left side serves as the main visual canvas using a clean, borderless set of axes with a fixed 120 by 80 coordinate space. The right side houses a dedicated control dashboard containing all the sliders, buttons, and readouts.

Since the game relies on standard MATLAB UI controls instead of App Designer's object-oriented framework, all core game data—such as the ball's position, velocity, level layout, score, and active particles—is stored directly in the main function's workspace. This approach allows every nested callback function (whether triggered by a slider, button, or mouse movement) to seamlessly share and update the exact same variables, keeping the entire application self-contained.



Figure 1- Overview of the GUI layout: The dashboard controls on the right are drawn to scale using their exact coordinates from the code, while the playing field on the left is an example layout rather than a live screenshot.

Instead of relying on a MATLAB timer object, the entire game is driven by a single while loop that runs continuously for as long as the window remains open (Figure 2). On every pass, the loop updates the physics only when the ball is actually in motion, though it constantly refreshes the dashboard readouts, steps through particle animations, and redraws the playing field using an hgtransform group.

To keep performance smooth, a brief drawnow limitrate; pause(0.02) combination sets a steady pace of around 50 frames per second, which prevents lagging and ensures MATLAB stays responsive to input events. Player actions such as tweaking sliders, swapping clubs, clicking buttons, or dragging the mouse, run as standard asynchronous callbacks. These callbacks simply update shared variables in the background, which the main loop picks up on its very next iteration.

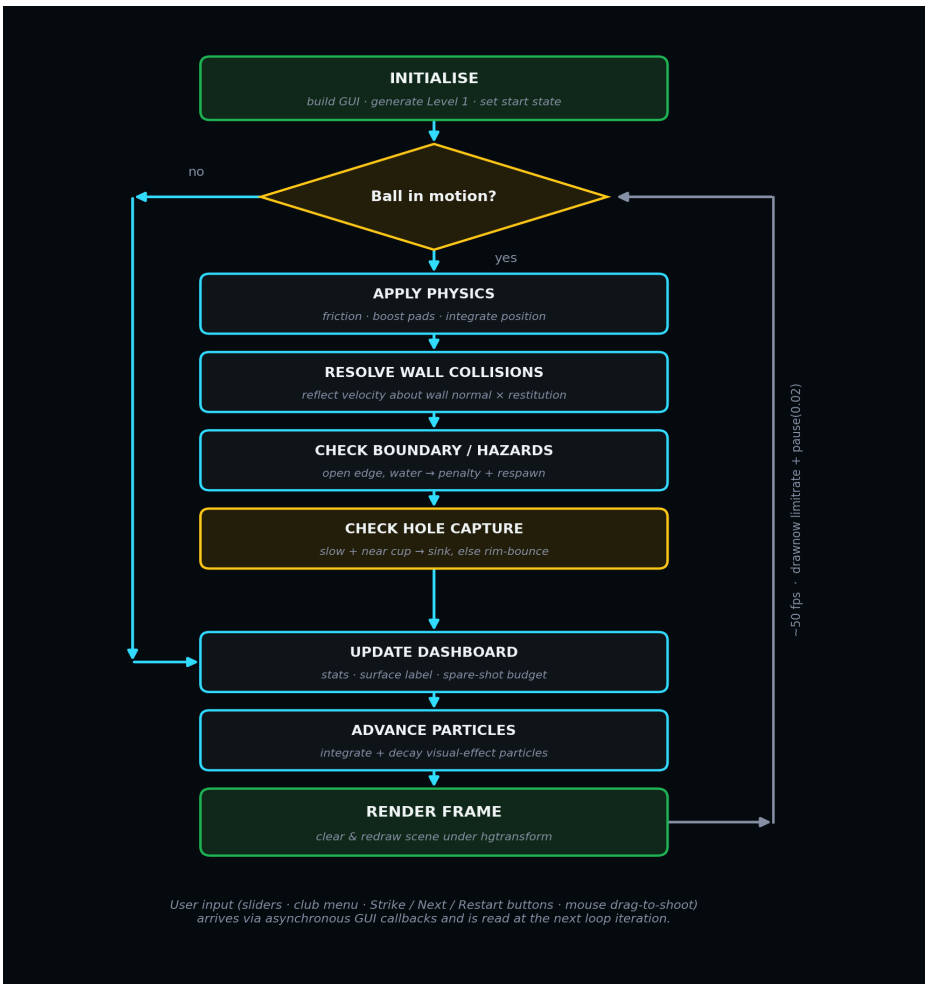


Figure 2. One iteration of the core game loop.

4. Physics Engine

The ball's movement relies on a simple step-by-step calculation—adding velocity to its position and applying friction to slow it down. Because that basic movement logic is so straightforward, most of the coding effort went into fine-tuning how surface friction interacts with the ball and handling realistic bounce physics when hitting walls.

Condition	Value	Notes
Fairway friction	0.965 / frame	Default rolling resistance across open grass.
Sand-trap friction	0.80 / frame	Drops to 0.90 (less draggy) if the Sand Wedge is selected.
Driver / Putter / Sand Wedge power	1.00 / 0.55 / 0.80	Multiplies the slider- or drag-based shot speed.
Wall restitution (e)	≈ 0.85 – 0.88	Per-wall coefficient; scales the reflected velocity.

To detect wall collisions, the game runs a simple circle-to-segment check by calculating the closest point on each wall segment relative to the ball's center. A hit is registered as soon as this distance drops below the ball's radius. To handle the bounce, the ball's incoming velocity vector is reflected across the wall's normal surface angle and scaled down by the wall's bounciness factor, or restitution coefficient, to simulate a realistic loss of energy on impact (Figure 3):

```
dot_p = ball_vx * norm_x + ball_vy * norm_y;
if dot_p < 0
    ball_vx = (ball_vx - 2 * dot_p * norm_x) * w(5);
    ball_vy = (ball_vy - 2 * dot_p * norm_y) * w(5);
end
```

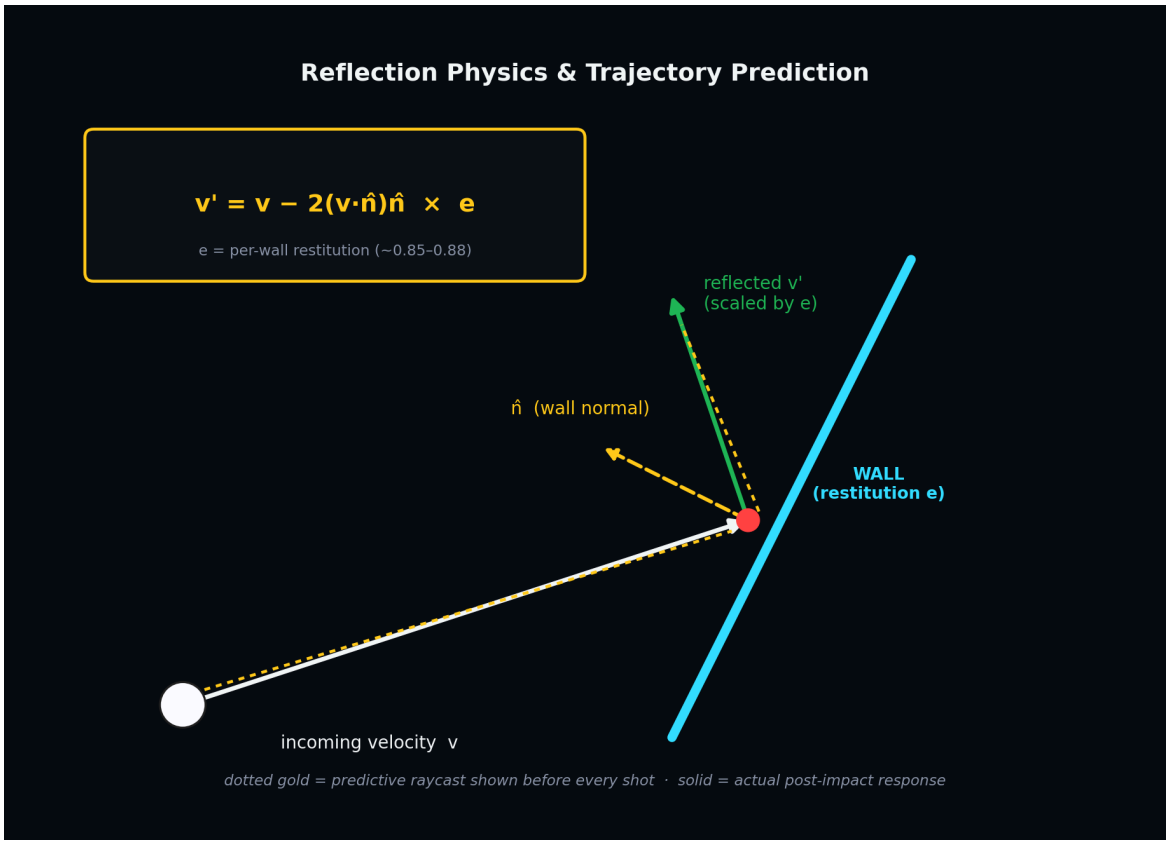


Figure 3. Vector reflection off a wall — the same formula drives the dotted trajectory-preview line, which raycasts the aim direction against every wall and re-applies the reflection once to preview a single bounce before the ball is struck.

5. Procedural Level Generation

Every course is created using the `generateProceduralLevel(N)` function, which takes the current level number (N) and builds a complete layout from scratch. As we progress, the game gradually ramps up the difficulty as par requirements shift, the target hole shrinks slightly, and more interior walls, sand traps, and water hazards are added to the map (Figure 4). The generator also randomly picks between one and three outer boundary edges to act as solid bounce walls, leaving the rest open as out-of-bounds hazards.

To keep the random layouts fair and playable, built-in clearance rules prevent impossible scenarios: sand traps aren't allowed within 15 units of the starting spot, and water hazards can't spawn within 18 units of the cup. This ensures you never start the level stuck in a trap or face a completely blocked hole.

```
lvl_data.par = 3 + floor((lvl - 1) / 3);
lvl_data.hole_radius = max(1.8, 3.6 - (lvl * 0.15));
active_indices = randperm(4, randi([1, 3]));   % which edges bounce
```

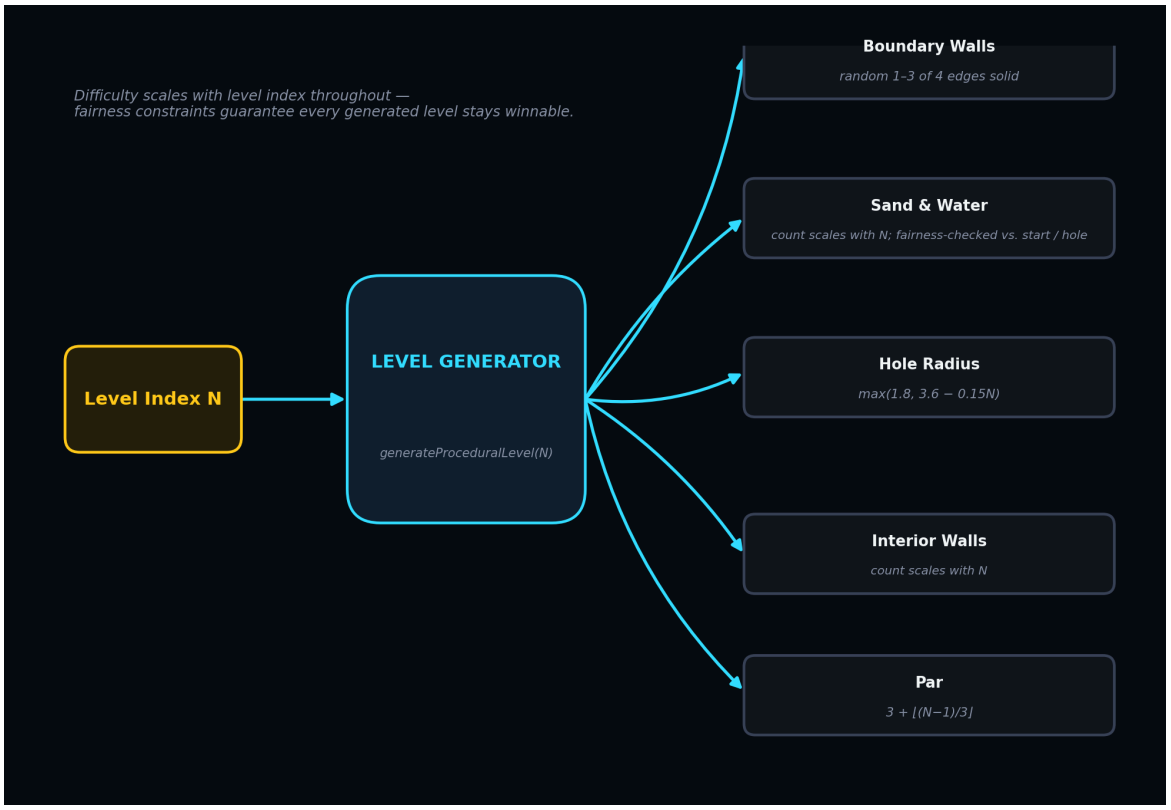


Figure 4. Inputs and outputs of the procedural level generator.

6. Controls & How to Run

Running the game is straightforward since it relies entirely on a standard, out-of-the-box MATLAB installation with no extra toolboxes required. Just save the code file as golf_streak_procedural.m, open MATLAB, and set your working directory to the folder where you saved the file. From there, you can launch the game either by typing golf_streak_procedural() directly into the Command Window or simply clicking the **Run** button in the Editor tab.

Input	Action
Aim-angle slider	Sets shot direction (0–360°) while the ball is stationary.
Shot-power slider	Sets shot strength (10–100%).
Club drop-down	Switches Driver / Putter / Sand Wedge — changes power multiplier and bunker friction.

Input	Action
Strike Ball button	Fires the shot using the current slider values.
Click-and-drag on the ball	Alternative aim: drag away from the target and release — drag distance sets power.
Next Level button	Advances to a new procedurally generated level once the hole is sunk.
Restart Game button	Resets the campaign to Level 1 with a fresh 10-shot budget.

7. Output Demonstration

The screenshots below represent some key features and moments of the game during playing it and moving on to the next level. Each figure shows the interface along with the game and briefly describes what's happening in the screenshot below each figure.

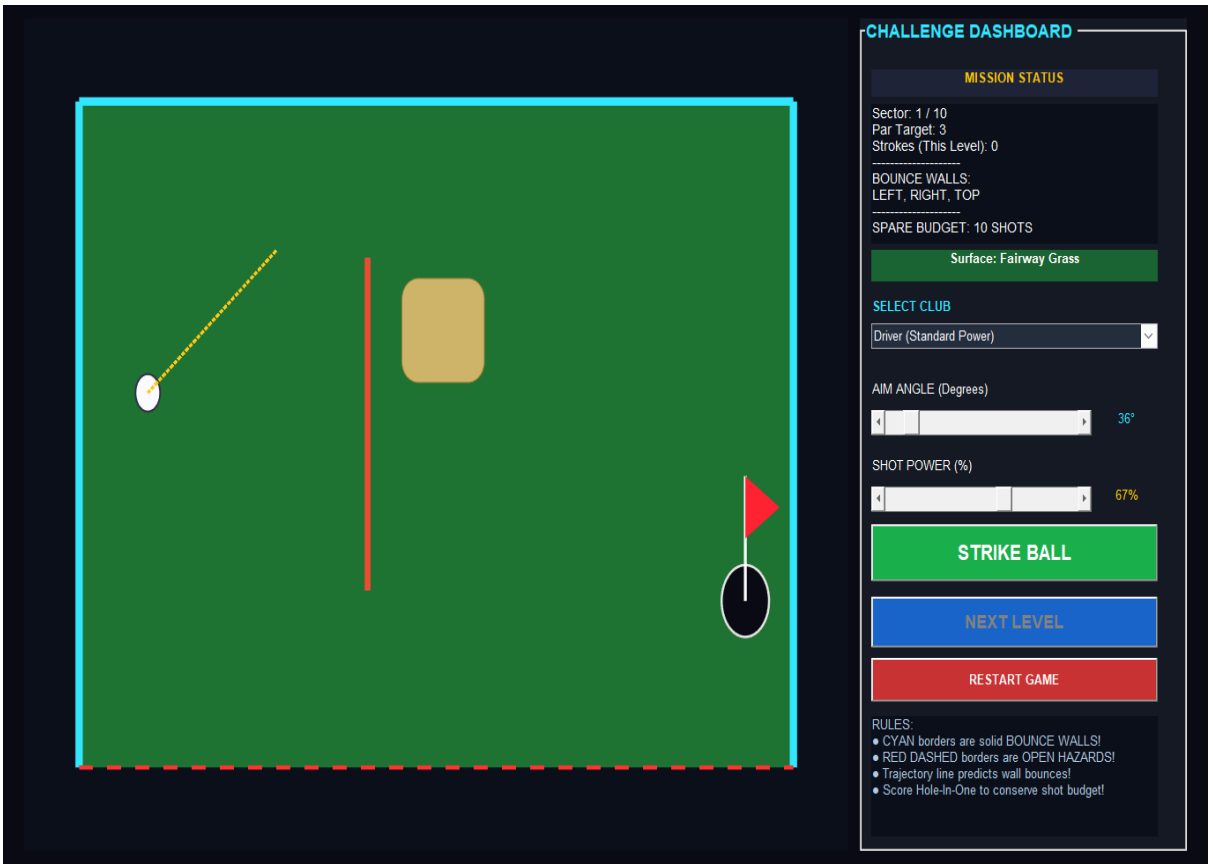


Figure 5. Opening view of Sector 1: dashboard on the right, ball and hole visible, trajectory-preview line showing before the first shot.



Figure 6. A shot in flight with the dotted trajectory preview showing a bounce off a boundary or interior wall.

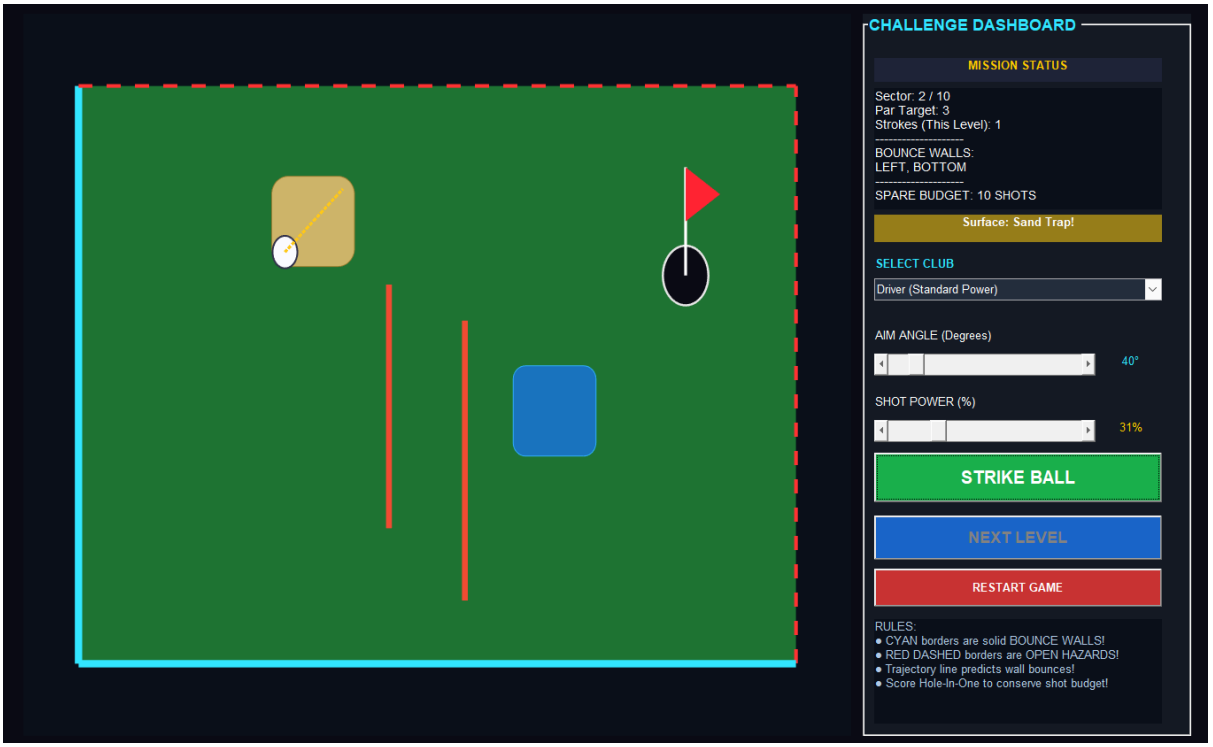


Figure 7. The dashboard's surface indicator reading **"Sand Trap!"** while the ball sits in a bunker.

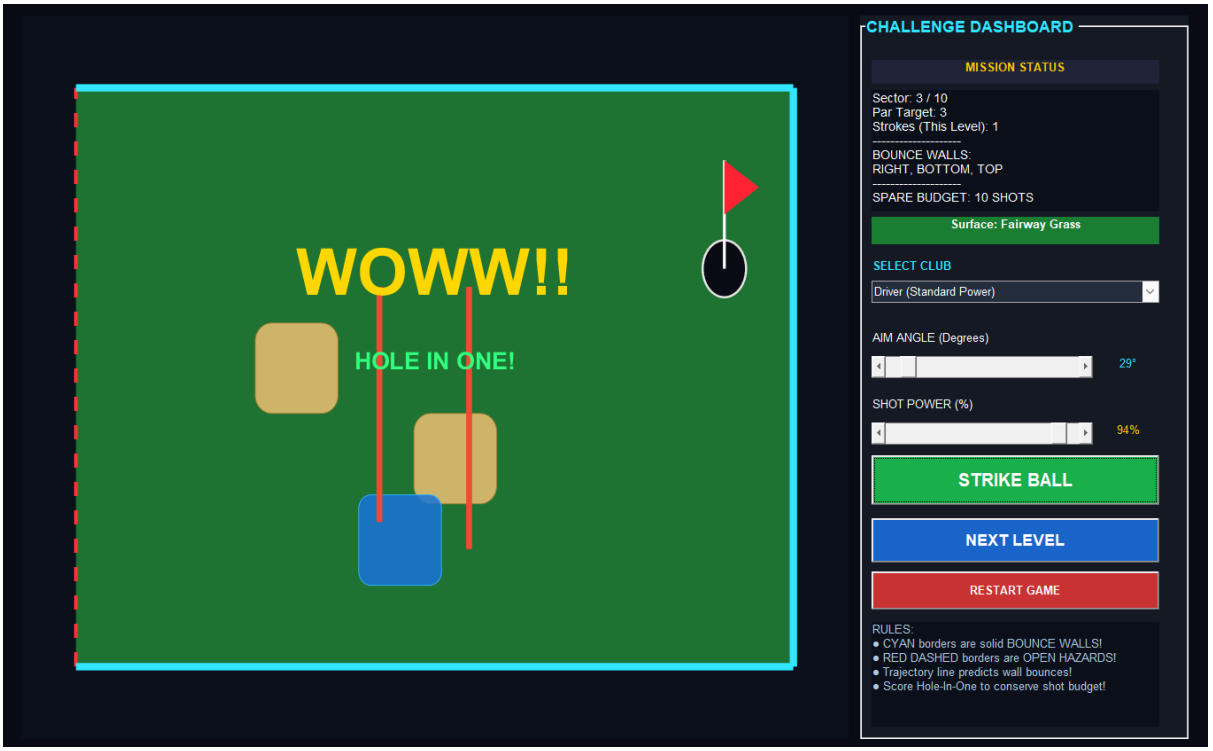


Figure 8. The **"HOLE IN ONE!"** celebration text after sinking a hole on the first stroke.

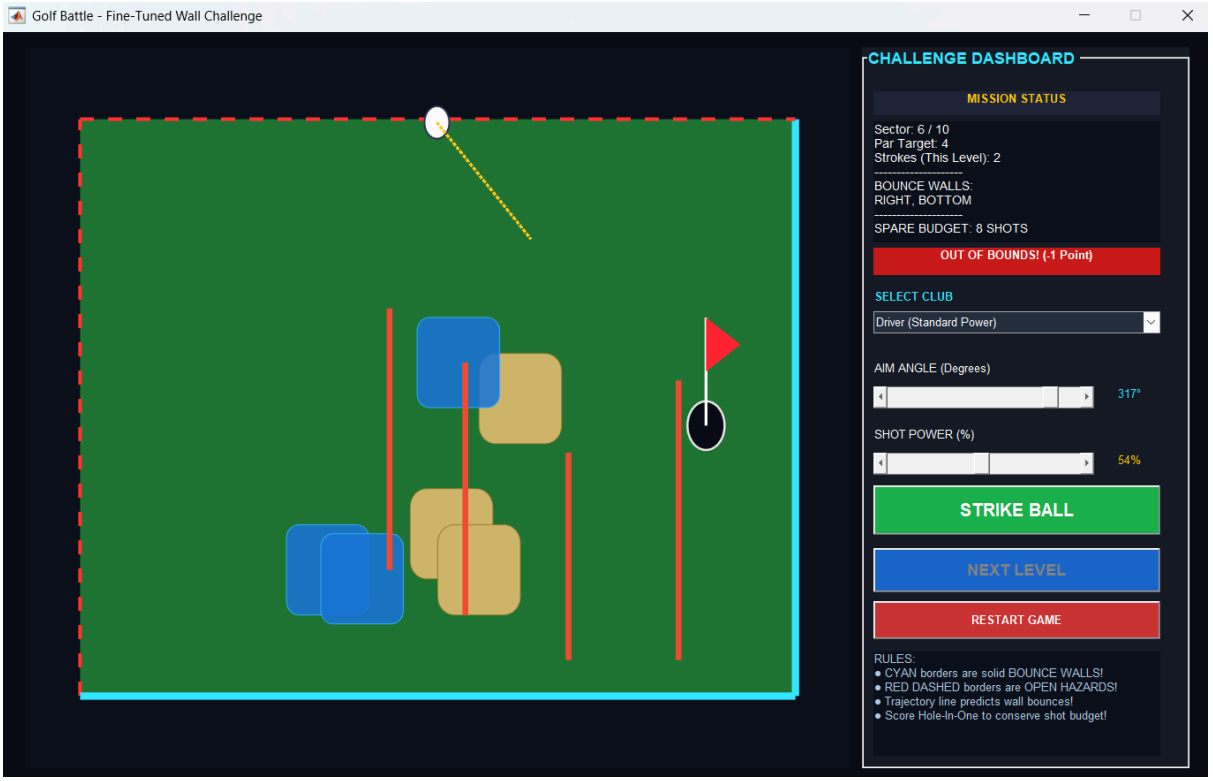


Figure 9. A later sector (Level 6 or higher) showing the active bounce-wall list and a reduced spare-shot budget.

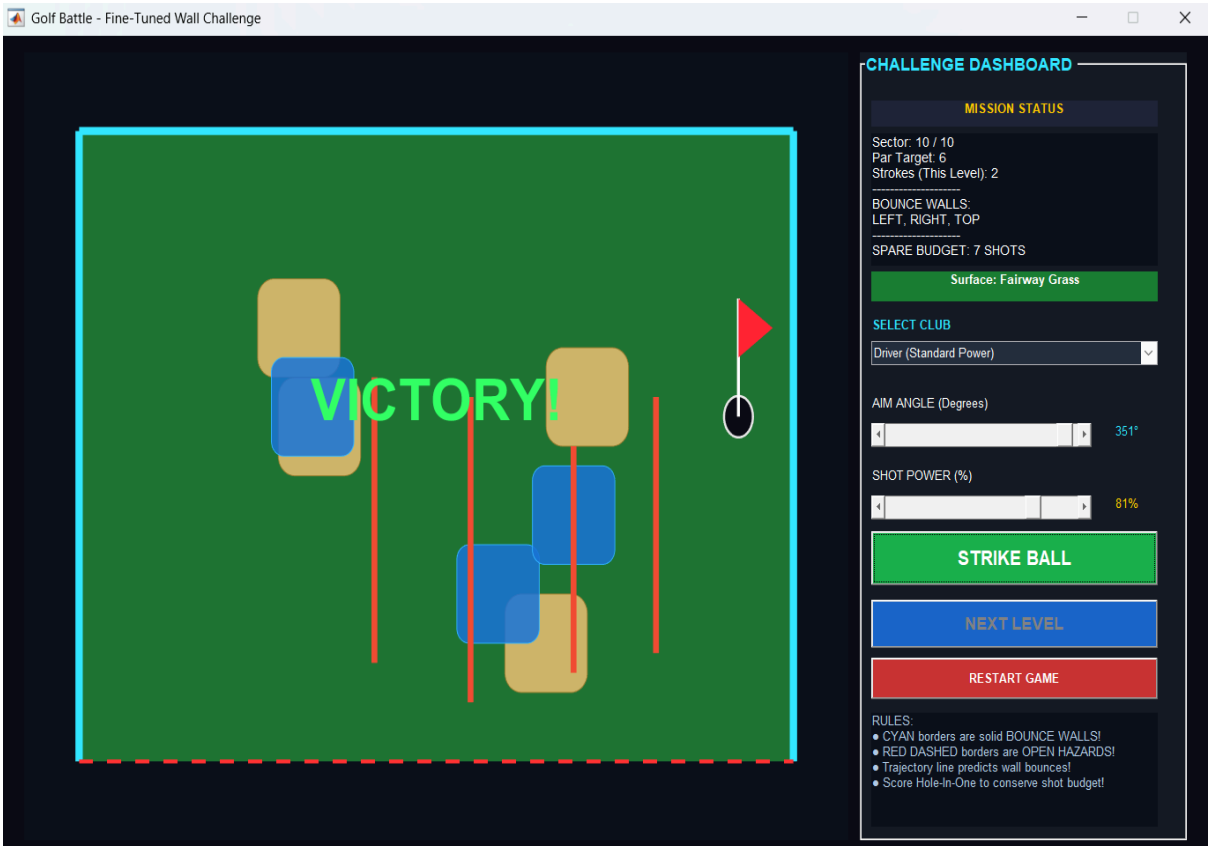


Figure 10. The end of campaign **VICTORY** or **GAME OVER** screen.

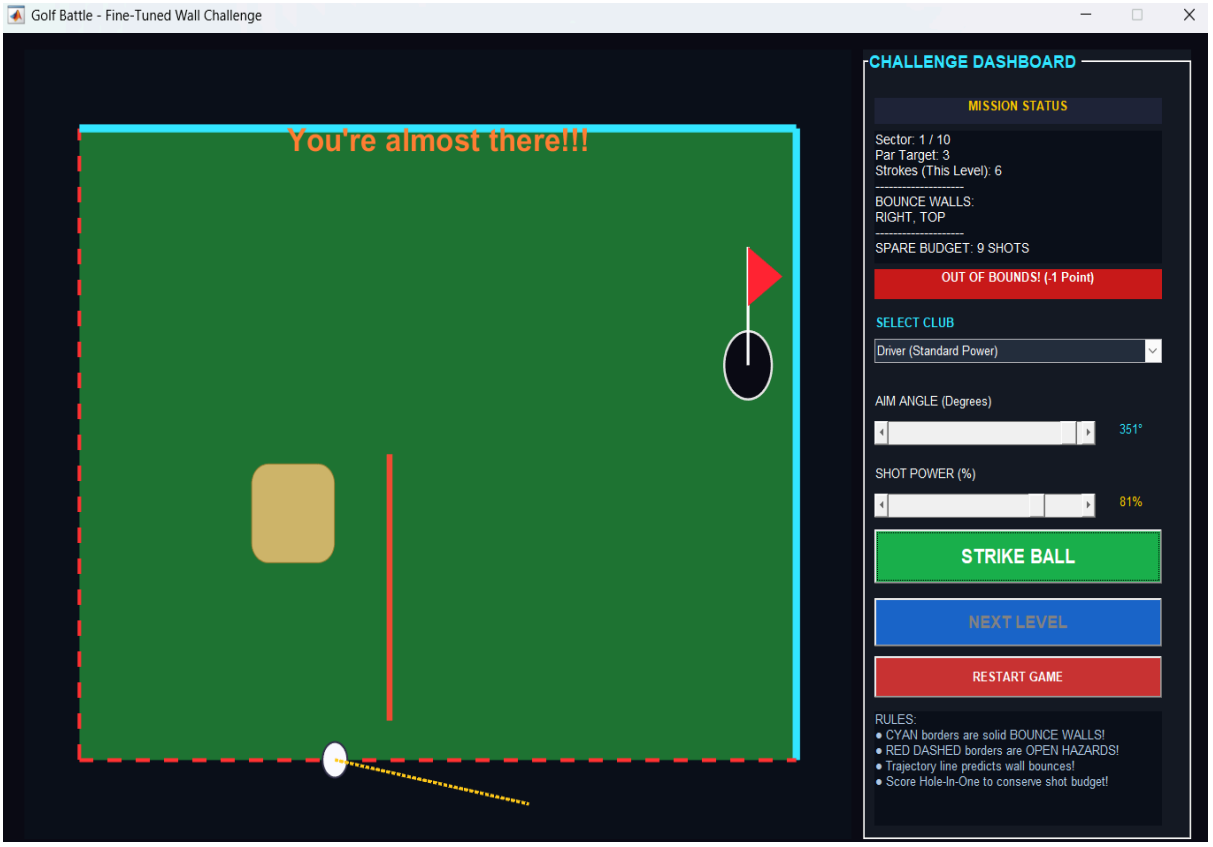


Figure 11. "You're almost there!!!" message after failing to score in 6 attempts.

8. Challenges & Design Decisions

- **Building custom 2D physics:** Writing realistic wall bounces and vector reflections from scratch without relying on a physics toolbox required a lot of trial-and-error vector math.
- **Ensuring level fairness:** Procedural generation can easily create broken layouts, so setting up strict buffer zones around starting points and holes was vital to keep every map beatable.
- **Balancing difficulty:** Fine-tuning how par targets, cup sizes, and hazard counts scale across all 10 levels took careful tweaking so the campaign feels progressively tougher without ever feeling unfair.
- **Keeping animation smooth:** Opting for a while loop with drawnow limitrate instead of a timer object sacrificed a tiny bit of timing precision, but it kept the code simple and made GUI callbacks instantly responsive

9. Possible Future Work

- **Audio feedback:** Adding crisp sound effects for ball strikes, wall reflections, and sinking a shot to make the game feel more lively.
- **Score saving:** Implementing local file storage to save high scores and track your best campaigns over time.
- **Dynamic hazards:** Introducing new mechanics like moving obstacles or wind forces that actively distort your shot preview.
- **Custom level editor:** Building a straightforward course editor so players can design and share their own sectors alongside the procedural ones.

10. Conclusion

Ultimately, **Golf Streak** demonstrates that MATLAB can go far beyond static data visualization and basic scripting. By combining custom 2D physics, a responsive event-driven interface, and a smart procedural level generator, the project evolves from a simple assignment into a fully interactive, replayable game. Building every system from scratch, from vector reflection math to real-time animation control, highlights just how flexible MATLAB's core graphics stack can be when pushed to deliver a smooth, engaging, and polished user experience.